Project Title:

TripLog Analyzer – A Python-Based Transportation Data Processing Tool

Course:

Introduction to Problem Solving & Programming (Python)

Domain:

Transportation & Logistics – Data Processing / Analyzer

Submitted By:

Name: *Pranjal Bhatnagar*

Roll No: 25BCE10653

Submitted To:

Dr J Sharmila Joseph

Academic Year:

2025–2026

2. Introduction

Transportation and logistics services such as buses, vans, cabs, and delivery vehicles generate a large number of trips every day. These trips are usually recorded in notebooks or basic spreadsheets without any proper analysis. As a result, it becomes difficult for operators to understand how many trips were completed, how much distance was covered, which vehicles are used the most, and how often delays occur.

The **TripLog Analyzer** project is a simple, Python-based tool that acts as a basic analytics system for transportation data. Instead of tracking expenses, it tracks **trips** taken by vehicles such as buses, vans, or taxis. Trip data is stored in a CSV file and processed using Python.

The program reads trip records from a CSV file and helps the user understand:

- How many trips were taken
- How much total distance was covered
- How trips are distributed across different vehicles
- How many trips were delayed

The main goal is to build a quick, terminal-based way to analyze transport/logistics data, keep it structured, and make the code easy to read, maintain, and extend.

---

3. Problem Statement

Small transport operators, college bus route coordinators, and local cab services often maintain trip logs in a very basic way—on paper or in simple Excel sheets. They rarely perform any systematic analysis on this data. Because of this, they lack answers to important questions such as:

- What is the total distance travelled by all vehicles?
- Which vehicles are used the most?
- How many trips face delays?
- Are delays increasing or decreasing over time?

Without a simple tool to **process and summarize** trip data, decision-making becomes guesswork.

Problem Statement:
To design and implement a Python-based command-line application that reads trip records from a CSV file, validates them, and generates meaningful analytics such as trip

counts, total distance, vehicle-wise usage, and delay statistics for small transportation/logistics setups.

---

4. Functional Requirements

The functional requirements describe **what the system should do**.

1. **Trip Data Loading**
    - The system shall load trip records from a CSV file (e.g., trips.csv).
    - Each record shall include: trip_id, date, vehicle_id, source, destination, distance_km, travel_time_min, delay_min.
2. **Data Validation**
    - The system shall validate basic fields (e.g., distance_km > 0).
    - Invalid or incomplete records shall be **skipped** without crashing the program.
3. **View All Trips**
    - The system shall allow the user to view all trips in a tabular format with columns like trip ID, date, vehicle, route, distance, travel time, and delay.
4. **Filter Trips by Vehicle**
    - The system shall allow the user to view only the trips of a specific vehicle_id.
5. **Search by Source/Destination**
    - The system shall allow searching trips by a text query present in the source or destination fields.
6. **View Delayed Trips**
    - The system shall allow the user to view only delayed trips, with an optional delay threshold (e.g., show trips delayed more than 5 minutes).
7. **Add New Trip**
    - The system shall allow the user to manually add a new trip entry via the terminal.
    - Newly added trips shall be appended to the in-memory list and saved back to the CSV file.
8. **Summary and Analytics**

- The system shall compute and display:
    - Total number of trips
    - Total distance travelled
    - Average distance per trip
    - Number and percentage of delayed trips
    - Trips per vehicle (vehicle-wise usage)

9. **Menu-Driven Interaction**
    - The system shall provide a simple main menu with options such as:
        - Add new trip
        - View trips
        - Show summary
        - Quit

---

5. Non-functional Requirements

Non-functional requirements describe **how** the system should behave.

1. **Usability**
    - The system shall provide a clear, text-based menu so that non-technical users can easily navigate and perform operations.
2. **Reliability**
    - The system shall handle invalid and missing data gracefully, skipping bad records without terminating unexpectedly.
3. **Performance**
    - The system shall be able to process at least a few thousand trip records without noticeable delay on a normal computer.
4. **Maintainability**
    - The code shall be organized into logical functions (and optionally modules) so that future changes (e.g., adding new analytics) can be implemented easily.
5. **Portability**

- The system shall run on any platform where Python 3 is installed (Windows, Linux, macOS) without requiring external libraries.

6. **Resource Usage**
    - The program shall use only standard Python libraries such as csv and os, avoiding heavy dependencies.

---

6. System Architecture

The TripLog Analyzer follows a **modular, layered architecture**, separating data loading, processing, and presentation.

6.1 Layers / Components

1. **User Interface Layer (CLI)**
    - Responsible for showing menus, taking user input, and printing outputs.
    - Implemented mainly in main() and helper functions.
2. **Data Layer**
    - Responsible for loading and saving trip records from/to trips.csv.
    - Uses Python's csv module.
    - Functions like load_trips() and save_trips().
3. **Processing / Analytics Layer**
    - Responsible for computing all statistics and summaries.
    - Functions like print_summary() and filters for delayed trips or vehicle-wise trips.
4. **Reporting / Display Layer**
    - Responsible for formatting the output for the terminal.
    - Includes functions that display tables of trips and summaries in a clean way.

6.2 High-Level Architecture Diagram

---

7. Design Decisions & Rationale

1. **CSV as Storage Format**
    - Chosen because it is simple, human-readable, and easy to edit.
    - Works well with Python's built-in csv module.
    - No need for a database for a small project.
2. **Command-Line Interface (CLI)**
    - Easy to implement using the standard input/output.

- Focus remains on **problem solving and logic**, not GUI design.
- Works on any system with a terminal.

3. **Modular Code Structure**
   - Separate functions for:
     - Loading data
     - Viewing and filtering trips
     - Adding new trips
     - Summarizing data
   - This improves readability, testing, and maintenance.
4. **Use of Standard Libraries Only**
   - Keeps installation simple for evaluators.
   - Avoids dependency issues.
   - Shows that powerful solutions are possible even with the standard library.
5. **Menu-Driven Design**
   - Makes the program more approachable to non-technical users.
   - Clear mapping between each menu option and one main feature.

---

8. Implementation Details

Language: Python 3.x
Main Modules / Functions: all in a single .py file or separated as needed.

8.1 Data Representation

Each trip is represented as a Python dictionary:

```
{
  "trip_id": 1,
  "date": "2025-11-20",
  "vehicle_id": "BUS01",
  "source": "Campus",
  "destination": "CityCenter",
```

"distance_km": 15.5,

"travel_time_min": 40,

"delay_min": 5

}

All trips are stored in a list: trips = [trip1, trip2, ...].

8.2 Key Functions (Example)

- load_trips()
    - Reads trip records from trips.csv using csv.DictReader.
    - Converts numeric fields like distance_km, travel_time_min, and delay_min.
- save_trips(trips)
    - Writes the current list of trips back to trips.csv using csv.DictWriter.
- print_trips_list(trips)
    - Displays all trips in a formatted table.
- view_trips(trips)
    - Provides sub-menu for:
        - View all trips
        - View by vehicle
        - View delayed trips
        - Search by source/destination
- add_trip_flow(trips)
    - Interacts with the user to enter a new trip.
    - Validates numeric fields.
    - Appends to the list and calls save_trips().
- print_summary(trips)
    - Computes:
        - Total trips
        - Total distance

        - Average distance
        - Delayed trips and percentage
        - Trips per vehicle
- main()
    - Displays the main menu:
        - Add new trip
        - View trips
        - Summary
        - Quit

---

9. Testing Approach

The project is tested using **manual test cases** focused on typical and edge scenarios.

9.1 Test Categories

1. **File Handling Tests**
    - Case 1: Valid trips.csv with correctly formatted rows → all trips should load.
    - Case 2: Missing trips.csv → no crash, show 0 trips.
    - Case 3: Some rows with invalid distance or missing fields → these rows are skipped.
2. **Add Trip Tests**
    - Enter valid numeric values → trip is saved and appears in subsequent runs.
    - Enter negative distance or invalid numbers → rejected with error message.
3. **View & Filter Tests**
    - View all trips → all valid records displayed.
    - View trips by an existing vehicle → only relevant records shown.
    - View trips by non-existing vehicle → empty table with no errors.
    - Search by source/destination text → check partial and case-insensitive matches.

4. **Delay Tests**
    - Set delay threshold to 0 → all delayed trips appear.
    - Set delay threshold higher → only heavily delayed trips shown.
5. **Summary Tests**
    - Verify that total distance equals the sum of distances from all trips.
    - Verify delayed percentage is delayed_trips / total_trips * 100.

---

10. Challenges Faced

- **Input Validation:**
  Ensuring that the program does not crash when the user enters invalid text instead of numbers required careful use of try-except blocks.
- **Data Cleaning:**
  Handling bad rows in the CSV file (missing fields, wrong types) required extra validation before adding trips to the list.
- **Designing a Simple but Useful Menu:**
  Deciding which options to include without overcomplicating the interface took some trial and error.
- **Balancing Features and Simplicity:**
  It was important to avoid overengineering the project (e.g., using databases or heavy libraries) and stay aligned with the course level.

---

11. Learnings & Key Takeaways

- Gained hands-on experience with **file handling in Python** using the csv module.
- Understood how to represent real-world data (trips) using **lists and dictionaries**.
- Learned how to design a **menu-driven program** with clear user flow.
- Improved skills in **input validation** and error handling.
- Realized the importance of **modular code** for readability and future changes.
- Saw how even simple analytics (counts, sums, averages) can provide real value in a practical domain like transportation.

---

12. Future Enhancements

Some planned or possible improvements for TripLog Analyzer:

1. **Graphical User Interface (GUI)**
    - Build a simple GUI using Tkinter or a web-based interface using Flask.
2. **Advanced Analytics**
    - Average speed per trip (distance/time).
    - Busiest routes (source-destination pairs with maximum trips).
    - Weekly/monthly trend analysis.
3. **Database Support**
    - Store trips in SQLite or MySQL instead of CSV for better scalability.
4. **Exporting Reports**
    - Export summary and vehicle-wise reports to PDF or Excel.
5. **User Roles**
    - Add different access levels (e.g., admin vs viewer).